JIRA – PROJECT MANAGEMENT AND BUG TRACKING TOOL

Basic Concepts and Important Points :

1. What is Jira?

Jira is a work management and issue-tracking tool developed by Atlassian. It helps teams plan, assign, track, and manage work throughout a project.

2. Main Uses of Jira

- Tracking tasks, bugs, and user stories.
- Managing project backlogs and sprints.
- Assigning work to team members.
- Tracking issue status and progress.
- Supporting Agile and Scrum practices.
- Generating reports to monitor project progress

3. Important Jira Terms

1. Project/Space

A project or space is used to organize related work in Jira. It provides a place to manage issues, assign work to team members, and track progress. The terminology and available features depend on the Jira product and interface.

- A project can contain different types of issues, such as stories, tasks, and bugs.
- Team members can collaborate and monitor work within the project.
- Project settings control certain configurations, permissions, and workflows.

Example: A Banking Application project can contain issues related to login, payments, and account management.

2. Issue

An issue is an individual work item tracked in Jira. It can represent a requirement, a task, a bug, or another type of work.

- Every issue has a summary describing the work.
- It can contain a description, issue type, status, priority, assignee, and other fields.
- Issues have unique keys, such as BANK-101, which help users identify and access them.
- Issues can include comments, attachments, links, and activity history.

Example: BANK-101 – Fix login validation error can be created as a Bug issue.

3. Epic

An epic represents a large body of work that can be divided into smaller, related issues. It helps organize and track progress on a major feature.

Example: User Authentication, which includes login, registration, and password reset.

4. Story

A story describes a requirement from the user's perspective, explaining what the user wants and why.

Example: As a customer, I want to reset my password so that I can regain access to my account.

5. Task

A task represents a specific piece of work that needs to be completed. It can be assigned to a team member and tracked through a workflow.

Example: Prepare test data for the login module.

6. Bug

A bug represents a defect or unexpected behavior in an application, usually identified when the actual result differs from the expected result.

Example: The application allows users to log in with an incorrect password.

7. Sub-task

A sub-task is a smaller unit of work created under a parent issue to divide the work into manageable activities. It can be assigned and tracked separately.

Example: Under the task "Test the Login Page," create sub-tasks for testing valid credentials and invalid credentials.

8. Assignee

The assignee is the person responsible for working on an issue. The issue can be reassigned when responsibility changes.

- An issue may be assigned to an individual team member, depending on project settings and permissions.
- The assignee is expected to work on the issue and update its progress.
- The assignee can be different from the reporter.
- Some issues may remain unassigned until a team member takes responsibility.

Example: A developer is assigned a bug related to incorrect login validation and is responsible for fixing it.

9. Reporter

The reporter is the person who created or reported an issue.

- The reporter provides information about the issue or requirement.
- In software testing, the reporter is often the tester who identifies a defect.
- The reporter and assignee can be different people.
- The reporter may provide additional details or answer questions during investigation.

Example: A tester discovers that login fails with valid credentials and creates a bug in Jira. The tester is the reporter, while the developer assigned to fix it is the assignee.

10. Priority

Priority indicates the relative importance or urgency of an issue and helps the team decide which work needs attention first.

- Common priority levels include Highest, High, Medium, Low, and Lowest, although the available levels may vary.
- Teams use priority to organize work and identify urgent issues.
- Priority is different from status: priority indicates importance, while status indicates the current stage of work.
- Priority is also different from severity, which describes the impact of a defect.

Example: A login failure affecting all users may be assigned High priority, while a minor alignment issue on a rarely used page may receive Low priority.

11. Status

Status indicates the current stage of an issue in its workflow. It helps team members understand whether the work is pending, ongoing, under review, or completed.

- Common statuses include To Do, In Progress, and Done.
- A workflow defines the available statuses and the transitions between them.
- Team members move issues through transitions as work progresses, subject to workflow rules and permissions.
- Different issue types may follow different workflows.
- Marking an issue as Done should reflect the team's completion criteria, not merely that someone has stopped working on it.

Example: A bug may move through the workflow:

To Do → In Progress → Ready for Testing → Testing → Done.

The actual statuses depend on the workflow configured in Jira.

12. Description

The description provides detailed information about an issue, including its purpose, requirements, or the work that needs to be completed.

- A clear description helps team members understand the issue without repeatedly asking for clarification.

- For stories, it can explain the user requirement and business need.
- For bugs, it can include reproduction steps, expected behavior, and actual behavior.
- Acceptance criteria can also be included in the description or maintained in a separate field, depending on the project setup.

Example: For a login bug, the description explains the steps followed, the error received, and the expected login behavior.

13. Acceptance Criteria

Acceptance criteria define the conditions a story or requirement must satisfy to be considered acceptable.

- They clarify what the team needs to implement and test.
- They help reduce misunderstandings between developers, testers, and stakeholders.
- They can be written as a checklist or in a Given–When–Then format.
- They provide a basis for verifying whether the completed functionality meets the requirement.

Example: For a login story:

- Users can log in with valid credentials.
- Invalid credentials are rejected.
- Appropriate error messages are displayed.
- Passwords are handled securely.

14. Labels

Labels are tags used to categorize issues and make related work easier to identify and search.

- An issue can have multiple labels.
- Labels can be used in searches, filters, and dashboards where applicable.
- They are useful for grouping issues across different modules or activities.
- Teams should use consistent label names to avoid creating unnecessary duplicates.

Example: Issues related to login testing may use labels such as login, regression-testing, and authentication.

15. Sprint

A sprint is a fixed-length period in Scrum during which the team works toward a specific goal.

- The Scrum team plans the sprint before starting it.
- Selected backlog items and the plan for delivering them form the sprint backlog.
- A sprint has a Sprint Goal and a defined duration.
- The team tracks progress during the sprint and reviews the outcome at the end.
- Sprint duration is consistent within a team's approach and is typically one month or less.

Example: A team plans a two-week sprint to implement login functionality, test authentication, and fix related defects.

16. Backlog

A backlog is an ordered list of work items that may be needed for the product or a planned iteration.

- The **Product Backlog** contains the broader set of product work and is ordered according to factors such as value, risk, and dependencies.
- The **Sprint Backlog** contains the work selected for a particular sprint and the plan for achieving the Sprint Goal.
- Backlog items can be clarified, prioritized, estimated, added, or removed as requirements change.
- Backlog refinement helps ensure that items are sufficiently understood for future planning.

Example: The product backlog may contain stories for login, registration, password reset, and payment processing. During sprint planning, the team selects appropriate items for the upcoming sprint.

JIRA – PROJECT MANAGEMENT AND BUG TRACKING TOOL

4. Agile and Scrum Basics

Agile: An iterative approach to software development that emphasizes adaptability, collaboration, and frequent delivery.

Scrum: An Agile framework that organizes work into iterations called sprints.

Product Backlog: An ordered list of work items that may be needed for the product.

Sprint: A fixed-length period during which a Scrum team works toward a Sprint Goal.

Sprint Goal: The objective the team aims to achieve during a sprint.

Backlog Refinement: An ongoing activity to clarify, split, estimate, and prioritize backlog items.

Sprint Planning: The event in which the Scrum team plans the work and establishes the Sprint Goal.

5. Jira Boards

Scrum Board: Supports sprint planning and tracking work during sprints.

Kanban Board: Visualizes work as it moves through workflow stages and supports continuous flow.

A board typically displays columns representing statuses or workflow stages.

6. Jira Workflow

A **workflow** defines the sequence of statuses and transitions an issue passes through from creation to completion.

Example:

To Do → In Progress → In Review → Done

Important points:

- Each status represents the current stage of an issue.
- A transition is an action that moves an issue from one status to another.
- Workflows help teams track the progress of tasks, stories, and bugs.
- Jira provides workflows that can be configured according to project requirements.

- We can also modify a workflow by clicking **Edit workflows** (if available and permitted) and adding or configuring the required statuses and transitions.
- Workflow editing options depend on the Jira project type, permissions, and configuration.

Example: In a software testing project, we can configure a workflow such as:

To Do → In Progress → Ready for Testing → Testing → Done

This allows the team to track an issue through development and testing until completion.

7. Important Jira Features

1. Filters

Filters are used to search for and display specific issues based on conditions such as status, assignee, priority, issue type, or project. They help us find the required issues quickly without checking every issue manually.

- Jira provides basic search to filter issues using fields and conditions.
- We can save frequently used searches as filters and access them later.
- Saved filters can be shared with other team members, depending on permissions.
- Jira Query Language (JQL) can be used to create more advanced searches.

Example: To find all high-priority bugs that are still open, we can use JQL:

project = BANK AND issuetype = Bug AND priority = High AND status != Done

What is JQL?

JQL stands for Jira Query Language. It is used to search for issues by combining fields, operators, and conditions.

Common JQL fields include project, assignee, status, issuetype, and priority.

Common operators include =, !=, IN, IS, and ORDER BY.

Example:

assignee = currentUser() AND status = "In Progress" ORDER BY priority DESC

This finds issues assigned to the current user that are In Progress and sorts them by priority in descending order.

2. Dashboards

A dashboard provides a single view of important project information. It helps team members monitor work, identify pending issues, and understand project progress without opening every issue individually.

- Dashboards contain gadgets that display information in different formats.
- We can add gadgets such as Filter Results, Pie Chart, Created vs Resolved Chart, and Assigned to Me.
- We can customize dashboards according to our role and requirements.
- Dashboards can be shared with other users or groups, depending on permissions.
- A dashboard can display information from saved filters.

Example: A tester can create a dashboard showing assigned bugs, high-priority issues, and issues that are still unresolved.

3. Reports

Reports help teams analyze work progress and understand how a sprint or project is performing.

- A Sprint Report shows completed and incomplete work for a sprint.
- A Burndown Chart shows the remaining work over the course of a sprint.
- A Velocity Chart shows how much work a Scrum team completes across sprints.
- A Cumulative Flow Diagram helps visualize how work is distributed across workflow stages over time.

Example: A Scrum team can use a Burndown Chart to check whether the remaining sprint work is decreasing as expected.

The available reports depend on the project type and Jira configuration.

4. Labels

Labels are tags used to categorize issues and make related work easier to find.

- We can add one or more labels to an issue.
- Labels can be used in searches and filters.
- They are useful for grouping issues across different modules or activities.
- Teams should use consistent label names to avoid duplicate or confusing labels.

Example: We can use labels such as login, regression-testing, and backend to identify related issues.

5. Components

Components are used to divide a project into smaller functional areas or modules.

- They help organize issues according to the part of the product they belong to.
- A component can have a component lead and a default assignee, depending on the project setup.
- Components are useful when different team members handle different modules.
- Components are different from labels because they are managed as project-level entities.

Example: A banking application project may contain components such as Login, Payments, and Account Management.

6. Versions and Releases

Versions help teams organize work associated with a particular release of a product.

- A version can represent a planned or completed software release.
- Issues can be associated with a version using fields such as Fix Version.
- Teams can track which bugs and features are planned for a release.
- Version information helps with release planning and tracking.

Example: If a team is preparing version 2.0 of an application, it can associate the bugs and features planned for that release with version 2.0.

7. Comments

Comments allow team members to communicate directly within an issue.

- Team members can share updates, ask questions, and clarify requirements.
- Testers can provide additional information when reporting bugs.
- Developers can explain the progress of a fix.
- Comments keep discussions connected to the relevant issue.

Example: A tester can comment, "The issue is reproducible on Chrome when the password field is left empty."

8. Attachments

Attachments allow users to add supporting files to Jira issues.

- We can attach screenshots, documents, logs, and other supported files.
- Testers can attach screenshots showing the actual result of a failed test.
- Developers can use logs and other evidence to investigate defects.
- Attachments make bug reports easier to understand and reproduce.

Example: A tester reports that clicking the Login button produces an error and attaches a screenshot of the error message.

9. Notifications

Notifications inform users about relevant changes and activities in Jira.

- Users may receive notifications when they are assigned an issue, mentioned in a comment, or involved in an issue update.
- Notifications help team members stay informed about work that requires their attention.
- Notification behavior depends on project settings, notification schemes, and personal preferences.

Example: When a bug is assigned to a developer, Jira may send the developer a notification about the assignment.

10. Issue Linking

Issue linking is used to establish relationships between different issues.

- It helps teams understand dependencies between tasks and bugs.
- Common link types include blocks, is blocked by, relates to, and duplicates.
- Issue links help teams identify work that depends on other work.
- Available link types may depend on the Jira configuration.

Example: If implementing a payment feature depends on completing user authentication, the related issues can be linked to show that dependency.

11. Activity History

Activity history helps users understand the changes made to an issue over time.

- It can show changes to fields such as status, assignee, priority, and description.
- It helps teams understand when an issue was updated and what changed.
- It is useful for reviewing issue progress and investigating unexpected changes.

Example: A tester can check the issue history to see when a bug moved from In Progress to Ready for Testing.

12. Automation

Automation allows repetitive work to be performed automatically using rules instead of manual actions.

An automation rule generally contains a trigger, optional conditions, and one or more actions.

- **Trigger:** Defines what starts the rule.
- **Condition:** Checks whether specific requirements are satisfied.
- **Action:** Defines what Jira should do when the conditions are met.

Example: When an issue transitions to Done, an automation rule can add a comment or notify a relevant team member.

Automation can reduce repetitive work and help maintain consistent processes. Available rules and actions depend on permissions and Jira configuration.

13. Permissions

Permissions determine what users are allowed to do in a Jira project or across Jira.

- They control actions such as viewing issues, creating issues, editing issues, and assigning work.
- Different users may have different permissions based on their roles.
- Administrative permissions may be required to change project settings or workflows.
- Permissions help protect project information and prevent unauthorized changes.

Example: A developer may be allowed to update assigned issues, while only an authorized administrator can modify certain workflow settings.

8. Important Things to Remember in Jira

1. Every issue has a unique issue key, such as JIRA-101, which helps us identify and track it.
2. **Assignee** is the person responsible for working on an issue, whereas **Reporter** is the person who created or reported it.
3. **Priority** indicates the importance or urgency of an issue, while **Status** shows its current stage of progress.
4. The **Product Backlog** contains the work that may be needed for the product, whereas the **Sprint Backlog** contains the work selected for a particular sprint along with the plan to achieve the Sprint Goal.
5. A **workflow transition** moves an issue from one status to another, such as To Do to In Progress.
6. An **Epic** represents a large body of work that can be divided into smaller issues, such as stories and tasks.
7. A **Sprint** is a fixed-length period in Scrum during which the team works toward a specific goal.

8. A **Scrum Board** supports sprint planning and tracking, while a **Kanban Board** helps visualize and manage the continuous flow of work.
9. **Backlog Refinement** is used to clarify requirements, split large items, estimate work, and prioritize backlog items.
10. **Acceptance Criteria** define the conditions that a user story must satisfy to be considered acceptable.
11. **Story Points** are used by teams to estimate the relative effort, complexity, and uncertainty of work. They do not directly represent hours.
12. **Jira permissions** determine which actions a user can perform, such as creating issues, editing workflows, or managing project settings.
13. Not every issue needs to follow the same workflow. Different issue types can use different workflows depending on the project's configuration.
14. Updating an issue's status helps team members understand its current progress and identify work that is pending or blocked.
15. Jira features and settings may differ depending on the project type, user permissions, and Jira configuration.

9. Simple Practical Example

Suppose a team is developing and testing a login page for a banking application. Jira can be used to organize the work and track its progress.

Project: Banking Application

Epic: User Authentication

Story: As a user, I want to log in securely so that I can access my account.

Tasks:

- Design the login page.
- Implement login functionality.
- Prepare test cases for login.

JIRA – PROJECT MANAGEMENT AND BUG TRACKING TOOL

Bug: The application allows users to log in with an incorrect password.

Assignee: The developer responsible for fixing the bug.

Reporter: The tester who reported the bug.

Priority: High, because the defect affects login security.

Workflow: To Do → In Progress → Ready for Testing → Testing → Done.

Sprint: The team selects the login-related stories and tasks for a sprint and works toward the Sprint Goal.

Acceptance Criteria:

- Users can log in with valid credentials.
- Invalid credentials are rejected.
- Appropriate error messages are displayed.
- The password is handled securely.

How Jira helps: The team can assign work, track issue status, discuss problems through comments, attach screenshots, and monitor progress until the work is completed.

In the next document, **Jira_Practice.Docx**, we will cover the practical aspects of Jira.

Thank you!